Teoria Współbieżności

Kajetan Frątczak 23.10.2025 r.

Laboratorium 03

Przetwarzanie potokowe

1. Treść zadań

Zadanie 1

1. Dany jest bufor, do którego producent może wkładać dane, a konsument pobierać. Napisać program, który zorganizuje takie działanie producenta i konsumenta, w którym zapewniona będzie własność bezpieczeństwa i żywotności.
Zrealizować program:

1. przy pomocy metod wait()/notify().
    a. dla przypadku 1 producent/1 konsument
    b. dla przypadku n1 producentów/n2 konsumentów (n1>n2, n1=n2, n1<n2)
    c. wprowadzić wywołanie metody sleep() i wykonać pomiary, obserwując zachowanie producentów/konsumentów

2. przy pomocy operacji P()/V() dla semafora:
    a. n1=n2=1
    b. n1>1, n2>1

Zadanie 2

2. Zaimplementować rozwiązanie przetwarzania potokowego (Przykładowe założenia: bufor rozmiaru 100, 1 producent, 1 konsument, 5 uszeregowanych procesów przetwarzających.) Od czego zależy prędkość obróbki w tym systemie ? Rozwiązanie za pomocą semaforów lub monitorów (dowolnie). Zrobić sprawozdanie z przetwarzania potokowego.

2. Argumentacja rozwiązania

Zadanie 1.1 (wait/notify)

Rozwiązanie tego podpunktu opiera się na monitorze Javy z metodami synchronized oraz mechanizmach wait() i notify(). Bufor przechowuje jeden element i flagę isFull, która określa stan bufora. W przypadku 1 producenta i 1 konsumenta wystarczy obudzić jeden wątek z użyciem notify(): producent czeka, gdy bufor pełny (isFull==true), zaś konsument czeka gdy bufor pusty (isFull==false). Bezpieczeństwo: zostało osiągnięte poprzez użycie słowa kluczowego synchronized na metodach put i get, co zapewnia, że tylko jeden wątek może w danej chwili wykonywać operację na buforze.

W przypadku n1 producentów i n2 konsumentów jest trochę gorzej - zamiast notify() używam notifyAll(), żeby obudzić wszystkie oczekujące wątki. W tej części dodałem także identyfikatory producentów/konsumentów dla obserwacji zachowania (przy n1!=n2 są błędy, które poprawiłem w kolejnym podpunkcie).

W podpunkcie c) poprawiłem liczbę instrukcji - w podpunkcie b) okazało się, że np. przy 2 producentach i 3 konsumentach: mamy 200 wytworzonych produktów i 300 produktów które konsumenci chcą kupić. Dodałem tu metodę sleep() i wprowadziłem system: liczba_iteracji_produkcji*n1 =liczba_iteracji_konsumpcji*n2, czyli w skrócie dla 2 producentów i 3 konsumentów mamy: 2*150 = 3*100, czyli 150 iteracji producentów i 100 iteracji konsumentów.

Zadanie 1.2 (semafor)

W tej wersji zadania 1 synchronizacja odbywa się za pomocą operacji P() i V() dla semafora. Korzystam z trzech semaforów: mutex (wartość początkowa = 1, ochrona sekcji krytycznej), empty (wartość początkowa = rozmiar_bufora, zlicza wolne miejsca), full (wartość początkowa = 0, zlicza zajęte miejsca).

Zadanie 2 (przetwarzanie potokowe z buforem)

Przetwarzanie potokowe polega na podziale zadania na szereg kolejnych, niezależnych etapów przetwarzania, pomiędzy którymi dane są przekazywane za pomocą buforów. Zaimplementowałem potok z jednym buforem współdzielonym, w którym “slot” przechodzi przez wszystkie etapy. Synchronizacja zapewnia, że dany etap nie przetwarza danych zanim poprzedni etap nie zakończył swojej pracy nad tym samym slotem. Zarówno wersja z monitorem (wait/notify), jak i z semaforami, realizuje ten sam schemat logiczny.

3. Rozwiązanie

Zadanie 1.1.a

```
class Producer extends Thread {
   private Buffer _buf;

   public Producer(Buffer _buf) {
       this._buf = _buf;
   }

   public void run() {
       for (int i = 0; i < 100; ++i) {
           _buf.put(i);
       }
   }
}
class Consumer extends Thread {
   private Buffer _buf;

   public Consumer(Buffer _buf) {
       this._buf = _buf;
   }

   public void run() {
       for (int i = 0; i < 100; ++i) {
           System.out.println(_buf.get());
       }
   }
}
class Buffer {
   private int item; // bufor przechowuje jeden element
   private boolean isFull = false; // flaga stanu bufora

   // synchronizacja przy pomocy wait() oraz notify()
   public synchronized void put(int i) {
       while (isFull) {
           try {
               wait(); // czekaj jeśli bufor jest pełny
           } catch (InterruptedException e) {
               Thread.currentThread().interrupt();
               return;
           }
       }
       item = i;
       isFull = true;
       System.out.println("Producer put: " + i);
       notify(); // powiadom konsumenta
   }
```

```java
    public synchronized int get() {
        while (!isFull) {
            try {
                wait(); // czekaj jeśli bufor pusty
            } catch (InterruptedException e) {
                Thread.currentThread().interrupt();
                return -1;
            }
        }
        int value  = item;
        isFull = false;
        System.out.println("Consumer got: " + value);
        notify(); // powiadom producenta
        return value;
    }
}
public class PKmon {
    public static void main(String[] args) {
        // podpunkt a)
        // 1 producent 1 konsument
        Buffer buffer = new Buffer();
        Producer producer = new Producer(buffer);
        Consumer consumer = new Consumer(buffer);

        producer.start();
        consumer.start();

        try {
            producer.join();
            consumer.join();
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
        }
    }
}
```

Zadanie 1.1.b

```java
class Producer extends Thread {
    private Buffer _buf;
    private int producerId;

    public Producer(Buffer _buf, int producerId) {
        this._buf = _buf;
        this.producerId =producerId;
    }

    public void run() {
        for (int i = 0; i < 100; ++i) {
            _buf.put(i, producerId);
        }
    }
}
class Consumer extends Thread {
    private Buffer _buf;
    private int consumerId;

    public Consumer(Buffer _buf, int consumerId) {
        this._buf = _buf;
        this.consumerId = consumerId;
    }

    public void run() {
        for (int i = 0; i < 100; ++i) {
            _buf.get(consumerId);
        }
    }
}
class Buffer {
    private int item; // bufor przechowuje jeden element
    private boolean isFull = false; // flaga stanu bufora

    // synchronizacja przy pomocy wait() oraz notify()
    public synchronized void put(int i, int producerId) {
        while (isFull) {
            try {
                wait(); // czekaj jeśli bufor jest pełny
            } catch (InterruptedException e) {
```

```java
                Thread.currentThread().interrupt();
                return;
            }
        }
        item = i;
        isFull = true;
        System.out.println("Producer " + producerId + " put: " + i);
        notifyAll(); // powiadom wszystkich czekających
    }

    public synchronized int get(int consumerId) {
        while (!isFull) {
            try {
                wait(); // czekaj jeśli bufor pusty
            } catch (InterruptedException e) {
                Thread.currentThread().interrupt();
                return -1;
            }
        }
        int value  = item;
        isFull = false;
        System.out.println("Consumer " + consumerId + " got: " + value);
        notifyAll(); // powiadom wszystkich czekających
        return value;
    }
}

public class PKmon {
    public static void main(String[] args) {
        // podpunkt b)
        // n1 producentów n2 konsumentów
        Buffer buffer = new Buffer();

        // n1>n2
        int n1 = 3; // liczba producentów
        int n2 = 2; // liczba konsumentów

        Producer[] producers = new Producer[n1];
        for (int i = 0; i < n1; i++) {
            producers[i] = new Producer(buffer, i);
            producers[i].start();
        }

        Consumer[] consumers = new Consumer[n2];
        for (int i = 0; i < n2; i++) {
            consumers[i] = new Consumer(buffer, i);
            consumers[i].start();
        }

        try {
            for (int i = 0; i < n1; i++) {
                producers[i].join();
            }
            for (int i = 0; i < n2; i++) {
                consumers[i].join();
            }
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
        }
    }
}
```

Zadanie 1.1.c (poprawione iteracje)

```java
class Producer extends Thread {
    private Buffer _buf;
    private int producerId;
    private long sleepTime;
    private long startTime;
    private int iterations;

    public Producer(Buffer _buf, int producerId, long sleepTime, long startTime, int iterations) {
        this._buf = _buf;
        this.producerId =producerId;
        this.sleepTime = sleepTime;
        this.startTime = startTime;
        this.iterations = iterations;
    }
```

```java
    public void run() {
        for (int i = 0; i < iterations; ++i) {
            try {
                long elapsed = System.currentTimeMillis() - startTime;
                System.out.printf("[%5d ms] Producer %d put: %d%n", elapsed, producerId, i);
                _buf.put(i, producerId);
                Thread.sleep(sleepTime);
            } catch (InterruptedException e){
                Thread.currentThread().interrupt();
                break;
            }
        }
        System.out.printf("Producer %d zakonczyl prace%n", producerId);
    }
}
class Consumer extends Thread {
    private Buffer _buf;
    private int consumerId;
    private long sleepTime;
    private long startTime;
    private int iterations;
    public Consumer(Buffer _buf, int consumerId,  long sleepTime, long startTime,  int iterations) {
        this._buf = _buf;
        this.consumerId = consumerId;
        this.sleepTime = sleepTime;
        this.startTime = startTime;
        this.iterations = iterations;
    }
    public void run() {
        for (int i = 0; i < iterations; ++i) {
            try {
                long value = _buf.get(consumerId);
                long elapsed = System.currentTimeMillis() - startTime;
                System.out.printf("[%5d ms] Consumer %d got: %d%n", elapsed, consumerId, value);
                Thread.sleep(sleepTime);
            } catch (InterruptedException e){
                Thread.currentThread().interrupt();
                break;
            }
        }
        System.out.printf("Consumer %d zakonczyl prace%n", consumerId);
    }
}

class Buffer {
    private int item; // bufor przechowuje jeden element
    private boolean isFull = false; // flaga stanu bufora
    private int putCount = 0;
    private int getCount = 0;

    // synchronizacja przy pomocy wait() oraz notify()
    public synchronized void put(int i, int producerId) {
        while (isFull) {
            try {
                System.out.printf("  [WAIT] Producer %d czeka (buffer pelny)%n", producerId);
                wait(); // czekaj jeśli bufor jest pełny
                System.out.printf("  [RESUME] Producer %d wznawia prace%n", producerId);
            } catch (InterruptedException e) {
                Thread.currentThread().interrupt();
                return;
            }
        }
        item = i;
        isFull = true;
        putCount++;
        notifyAll(); // powiadom wszystkich czekających
    }

    public synchronized int get(int consumerId) {
        while (!isFull) {
            try {
                System.out.printf("  [WAIT] Consumer %d czeka (buffer pusty)%n", consumerId);
                wait();
                System.out.printf("  [RESUME] Consumer %d wznawia prace%n", consumerId);
            } catch (InterruptedException e) {
                Thread.currentThread().interrupt();
                return -1;
            }
        }
```

```java
        int value  = item;
        isFull = false;
        getCount++;
        notifyAll(); // powiadom wszystkich czekających
        return value;
    }

    public synchronized void printStatistics() {
        System.out.println("\n=== STATYSTYKA BUFORA ===");
        System.out.println("Lacznie wstawiono: " + putCount);
        System.out.println("Lacznie pobrano: " + getCount);
    }
}

public class PKmon {
    public static void main(String[] args) {
        long startTime = System.currentTimeMillis();
        // podpunkt c)
        // n1 producentów n2 konsumentów

        // n1>n2
        int n1 = 3; // liczba producentów
        int n2 = 2; // liczba konsumentów

        long producerSleep = 100;
        long consumerSleep = 100;

        int totalItems = 300;
        int producerIterations = totalItems / n1;
        int consumerIterations = totalItems / n2;

        Buffer buffer = new Buffer();

        System.out.println("=== KONFIGURACJA ===");
        System.out.println("Liczba producentow: " + n1);
        System.out.println("Liczba konsumentow: " + n2);
        System.out.println("Sleep producenta: " + producerSleep + " ms");
        System.out.println("Sleep konsumenta: " + consumerSleep + " ms");
        System.out.println("=====================================\n");

        Producer[] producers = new Producer[n1];
        for (int i = 0; i < n1; i++) {
            producers[i] = new Producer(buffer, i, producerSleep, startTime, producerIterations);
            producers[i].start();
        }

        Consumer[] consumers = new Consumer[n2];
        for (int i = 0; i < n2; i++) {
            consumers[i] = new Consumer(buffer, i,  consumerSleep, startTime, consumerIterations);
            consumers[i].start();
        }

        try {
            for (int i = 0; i < n1; i++) {
                producers[i].join();
            }
            for (int i = 0; i < n2; i++) {
                consumers[i].join();
            }
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
        }

        long endTime = System.currentTimeMillis();
        long totalTime = endTime - startTime;
        buffer.printStatistics();
        System.out.println("Calkowity czas wykonania: " + totalTime + " ms");
        System.out.println("Wszystkie watki zakonczone");
    }
}
```

Zadanie 1.2

```java
class Producer extends Thread {
   private Buffer _buf;
   private int iterations;
   private int producerId;

   public Producer(Buffer _buf, int producerId, int iterations) {
       this._buf = _buf;
       this.producerId = producerId;
       this.iterations = iterations;
   }

   public void run() {
       for (int i = 0; i < iterations; ++i) {
           _buf.put(producerId, i);
       }
   }
}

class Consumer extends Thread {
   private Buffer _buf;
   private int consumerId;
   private int iterations;

   public Consumer(Buffer _buf, int consumerId, int iterations) {
       this._buf = _buf;
       this.consumerId = consumerId;
       this.iterations = iterations;
   }

   public void run() {
       for (int i = 0; i < iterations; ++i) {
           int value = _buf.get(consumerId);
           System.out.println("Konsument " + consumerId + " pobral: " + value);
       }
   }
}

class Buffer {
   private final int[] buffer;
   private int in = 0, out = 0;
   private Semaphore mutex = new Semaphore(1);
   private Semaphore full = new Semaphore(0);
   private Semaphore empty;

   public Buffer(int size) {
       buffer = new int[size];
       empty = new Semaphore(size);
   }

   public void put(int id, int value) {
       empty.P(); // P(empty)
       mutex.P(); // P(mutex)
       System.out.println("Producent " + id + " wlozyl: " + value);
       buffer[in] = value;
       in = (in + 1) % buffer.length;
       mutex.V(); // V(mutex)
       full.V(); // V(full)
   }

   public int get(int id) {
       full.P(); // P(full)
       mutex.P(); // P(mutex)
       int value = buffer[out];
       out = (out + 1) % buffer.length;
       mutex.V(); // V(mutex)
       empty.V(); // V(empty)
       return value;
   }
}

class Semaphore {
   private int value;

   public Semaphore(int value) {
       this.value = value;
   }
```

```java
    public synchronized void P() {
        while (value <= 0) {
            try {
                wait();
            } catch (InterruptedException e) {
                Thread.currentThread().interrupt();
            }
        }
        value--;
    }

    public synchronized void V() {
        value++;
        notify();
    }
}

public class PKmon {
    public static void main(String[] args) {
        Buffer buffer = new Buffer(100);
        int n1 = 3;
        int n2 = 2;
        int iter1 = 100;
        int iter2 = 150;

        Producer[] producers = new Producer[n1];
        Consumer[] consumers = new Consumer[n2];

        for (int i = 0; i < n1; i++) {
            producers[i] = new Producer(buffer, i, iter1);
            producers[i].start();
        }

        for (int i = 0; i < n2; i++) {
            consumers[i] = new Consumer(buffer, i, iter2);
            consumers[i].start();
        }

        try {
            for (Producer p : producers) {
                p.join();
            }
            for (Consumer c : consumers) {
                c.join();
            }
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
        }
    }
}
```

Zadanie 2 (wersja z wait/notify)

```java
// przetwarzanie potokowe z buforem - WERSJA Z MONITORAMI (wait/notify)
class PipelineBuffer {
    private final int[] buffer;
    private final int[] outs;
    private int in = 0;
    private final int size;

    // Warunki (conditions)
    private boolean[] dataReady;
    private boolean allEmpty;

    public PipelineBuffer(int size, int stages) {
        this.size = size;
        this.buffer = new int[size];
        this.outs = new int[stages + 1];
        this.dataReady = new boolean[stages + 1];
        this.allEmpty = true;
    }
    // PRODUCENT - wstawia dane
    public synchronized void put(int value) {
        // Czekaj dopóki bufor pełny
        while (!allEmpty) {
            try {
```

```java
            wait();  // Czeka i zwalnia lock
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
        }
    }

    int slot = in;
    in = (in + 1) % size;
    buffer[slot] = value;

    allEmpty = false;  // Bufor już nie pusty
    dataReady[0] = true;

    notifyAll();  // Powiadom wszystkie czekające wątki
    System.out.println("[PRODUCENT] → slot " + slot + " = " + value);
}

// PROCESOR - przetwarzanie w etapie
public void process(int stage) {
    // Czekaj na dane z poprzedniego etapu
    synchronized(this) {
        while (!dataReady[stage]) {
            try {
                wait();
            } catch (InterruptedException e) {
                Thread.currentThread().interrupt();
            }
        }

        // Odczyt z bufora
        int slot = outs[stage];
        outs[stage] = (outs[stage] + 1) % size;
        int val = buffer[slot];

        // Resetuj flagę dla tego etapu
        dataReady[stage] = false;

        val += (stage + 1) * 100;
        try {
            Thread.sleep((stage + 1) * 10);
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
        }

        // Zapis wyniku - znowu w synchronized
        synchronized(this) {
            buffer[slot] = val;
            dataReady[stage + 1] = true;  // Sygnalizuj następnemu etapowi
            notifyAll();
        }

        System.out.println("[ETAP " + stage + "] slot " + slot + " = " + val);
    }
}
// KONSUMENT - pobiera rezultat
public synchronized void get() {
    // Czekaj na dane z ostatniego etapu
    while (!dataReady[dataReady.length - 1]) {
        try {
            wait();
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
        }
    }

    int slot = outs[outs.length - 1];
    outs[outs.length - 1] = (outs[outs.length - 1] + 1) % size;
    int val = buffer[slot];

    // Resetuj flagę dla konsumenta
    dataReady[dataReady.length - 1] = false;

    // Jeśli to była ostatnia dana, możemy znowu pisać
    allEmpty = true;

    notifyAll();
    System.out.println("[KONSUMENT] ← slot " + slot + " = " + val);
}
}
```

```java
class Producer extends Thread {
   private final PipelineBuffer buf;
   private final int count;
   public Producer(PipelineBuffer b, int c) {
       buf = b;
       count = c;
   }
   public void run() {
       for (int i = 0; i < count; i++) {
           buf.put(i);
           try { Thread.sleep(20); } catch (InterruptedException ignored) {}
       }
       System.out.println("[PRODUCENT] Koniec");
   }
}

class ProcessingStage extends Thread {
   private final PipelineBuffer buf;
   private final int stage, count;
   public ProcessingStage(PipelineBuffer b, int s, int c) {
       buf = b;
       stage = s;
       count = c;
       setName("Stage-" + s);
   }
   public void run() {
       for (int i = 0; i < count; i++) buf.process(stage);
       System.out.println("[ETAP " + stage + "] Koniec");
   }
}

class Consumer extends Thread {
   private final PipelineBuffer buf;
   private final int count;
   public Consumer(PipelineBuffer b, int c) {
       buf = b;
       count = c;
   }
   public void run() {
       for (int i = 0; i < count; i++) buf.get();
       System.out.println("[KONSUMENT] Koniec");
   }
}

public class PKmon {
   public static void main(String[] args) {
       int bufferSize = 100, stages = 5, items = 50;
       System.out.println("=== PRZETWARZANIE POTOKOWE (MONITORY) ===\n");

       PipelineBuffer buf = new PipelineBuffer(bufferSize, stages);
       Producer prod = new Producer(buf, items);
       Consumer cons = new Consumer(buf, items);
       ProcessingStage[] proc = new ProcessingStage[stages];

       long start = System.currentTimeMillis();

       prod.start();
       for (int i = 0; i < stages; i++) {
           proc[i] = new ProcessingStage(buf, i, items);
           proc[i].start();
       }
       cons.start();

       try {
           prod.join();
           cons.join();
           for (ProcessingStage p : proc) p.join();
       } catch (InterruptedException e) { Thread.currentThread().interrupt(); }

       long end = System.currentTimeMillis();
       System.out.printf("\n=== WYNIKI ===\n");
       System.out.printf("Czas wykonania: %d ms\n", (end - start));
       System.out.printf("Przepustowosc: %.2f elem/s\n",
               (items * 1000.0 / (end - start)));
   }
}
```

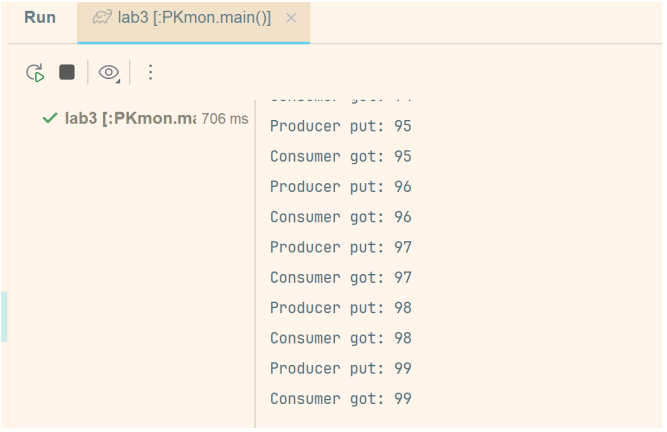
Zadanie 2 (wersja z semaforem)

```java
import java.util.LinkedList;

// przetwarzanie potokowe z buforem
class Semaphore {
   private int value;

   public Semaphore(int value) {
       this.value = value;
   }

   public synchronized void P() {
       while (value <= 0) {
           try {
               wait();
           } catch (InterruptedException e) {
               Thread.currentThread().interrupt();
           }
       }
       value--;
   }

   public synchronized void V() {
       value++;
       notifyAll();
   }
}

class PipelineBuffer {
   private final int[] buffer;
   private final int[] outs;
   private int in = 0;
   private final Semaphore mutex = new Semaphore(1);
   private final Semaphore empty;
   private final Semaphore[] ready;
   private final int size;
   public PipelineBuffer(int size, int stages) {
       this.size = size;
       buffer = new int[size];
       outs = new int[stages + 1];
       empty = new Semaphore(size);
       ready = new Semaphore[stages + 1];
       for (int i = 0; i < ready.length; i++) ready[i] = new Semaphore(0);
   }
   public void put(int value) {
       empty.P();  // Czekaj na wolne miejsce
       mutex.P();
       int slot = in;
       in = (in + 1) % size;
       buffer[slot] = value;
       mutex.V();
       ready[0].V();
       System.out.println("[PRODUCENT] → slot " + slot + " = " + value);
   }

   public void process(int stage) {
       ready[stage].P();  // Czekaj na dane z poprzedniego etapu

       int slot;
       int val;
       mutex.P();
       slot = outs[stage];
       outs[stage] = (outs[stage] + 1) % size;
       val = buffer[slot];  // Kopiuj wartość
       mutex.V();

       val += (stage + 1) * 100;
       try {
           Thread.sleep((stage + 1) * 10);  // Symulacja czasu pracy
       } catch (InterruptedException e) {
           Thread.currentThread().interrupt();
       }
       mutex.P();
       buffer[slot] = val;
       mutex.V();
       ready[stage + 1].V();  // Sygnalizuj następnemu etapowi
       System.out.println("[ETAP " + stage + "] slot " + slot + " = " + val);
   }
```

```java
    public void get() {
        ready[ready.length - 1].P();  // Czekaj na dane z ostatniego etapu
        mutex.P();
        int slot = outs[outs.length - 1];
        outs[outs.length - 1] = (outs[outs.length - 1] + 1) % size;
        int val = buffer[slot];
        mutex.V();
        System.out.println("[KONSUMENT] ← slot " + slot + " = " + val);
        empty.V();  // Zwolnij miejsce w buforze
    }
}

class Producer extends Thread {
    private final PipelineBuffer buf;
    private final int count;
    public Producer(PipelineBuffer b, int c) { buf = b; count = c; }
    public void run() {
        for (int i = 0; i < count; i++) {
            buf.put(i);
            try { Thread.sleep(20); } catch (InterruptedException ignored) {}
        }
        System.out.println("[PRODUCENT] Koniec");
    }
}

class ProcessingStage extends Thread {
    private final PipelineBuffer buf;
    private final int stage, count;
    public ProcessingStage(PipelineBuffer b, int s, int c) {
        buf = b; stage = s; count = c; setName("Stage-" + s);
    }
    public void run() {
        for (int i = 0; i < count; i++) buf.process(stage);
        System.out.println("[ETAP " + stage + "] Koniec");
    }
}

class Consumer extends Thread {
    private final PipelineBuffer buf;
    private final int count;
    public Consumer(PipelineBuffer b, int c) { buf = b; count = c; }
    public void run() {
        for (int i = 0; i < count; i++) buf.get();
        System.out.println("[KONSUMENT] Koniec");
    }
}

public class PKmon {
    public static void main(String[] args) {
        int bufferSize = 100, stages = 5, items = 50;
        System.out.println("=== PRZETWARZANIE POTOKOWE ===\n");

        PipelineBuffer buf = new PipelineBuffer(bufferSize, stages);
        Producer prod = new Producer(buf, items);
        Consumer cons = new Consumer(buf, items);
        ProcessingStage[] proc = new ProcessingStage[stages];

        long start = System.currentTimeMillis();

        prod.start();
        for (int i = 0; i < stages; i++) {
            proc[i] = new ProcessingStage(buf, i, items);
            proc[i].start();
        }
        cons.start();

        try {
            prod.join();
            cons.join();
            for (ProcessingStage p : proc) p.join();
        } catch (InterruptedException e) { Thread.currentThread().interrupt(); }

        long end = System.currentTimeMillis();
        System.out.printf("\n=== WYNIKI ===\n");
        System.out.printf("Czas wykonania: %d ms\n", (end - start));
        System.out.printf("Przepustowość: %.2f elem/s\n",
                (items * 1000.0 / (end - start)));
    }
}
```

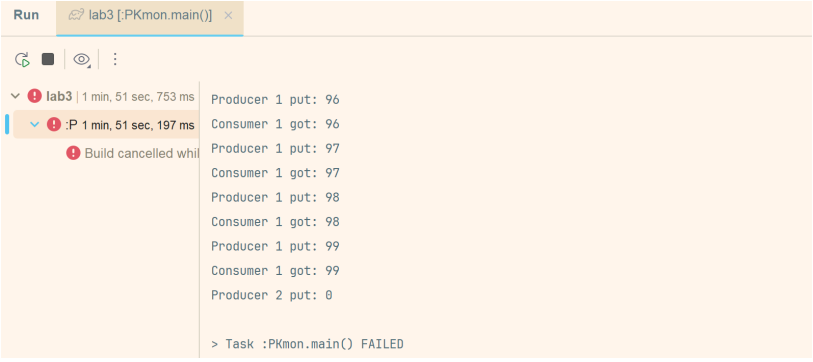
4. Wykresy, tabele, wyniki

1.1.a



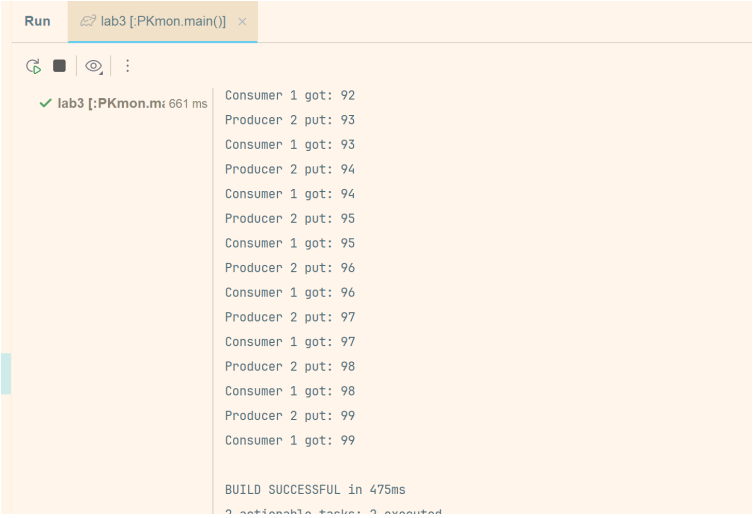
Rysunek 1: Wynik po uruchomieniu programu

1.1.b n1>n2 (błąd przez to, że mamy za mało iteracji konsumentów)



Rysunek 2: Wynik po uruchomieniu programu

1.1.b n1=n2



Rysunek 3: Wynik po uruchomieniu programu

1.1.b n1<n2 (błąd przez to, że mamy za mało iteracji producentów)

```
Run    lab3 [:PKmon.main()]

lab3 [:PKm 9 sec, 837 ms
:PKmon.ı 9 sec, 102 ms
Build cancelled whil

Producer 0 put: 95
Consumer 1 got: 95
Producer 0 put: 96
Consumer 1 got: 96
Producer 0 put: 97
Consumer 1 got: 97
Producer 0 put: 98
Consumer 1 got: 98
Producer 0 put: 99
Consumer 1 got: 99

> Task :PKmon.main() FAILED
```

Rysunek 4: Wynik po uruchomieniu programu

1.1.c n1>n2 (poprawione iteracje)

```
lab3 [:PKmon.main()]: Building: ':PKmon.main()'  22 sec
21 sec
  [RESUME] Producer 0 wznawia prace
[10047 ms] Producer 1 put: 65
  [WAIT] Producer 1 czeka (buffer pelny)
[10049 ms] Producer 2 put: 67
  [RESUME] Producer 1 wznawia prace
[10049 ms] Consumer 1 got: 67
  [WAIT] Producer 2 czeka (buffer pelny)
Consumer 0 zakonczyl prace
[10148 ms] Producer 0 put: 68
  [WAIT] Producer 0 czeka (buffer pelny)
[10150 ms] Producer 1 put: 66
  [WAIT] Producer 1 czeka (buffer pelny)
Consumer 1 zakonczyl prace
```

Rysunek 5: Wynik po uruchomieniu programu

1.1.c n1=n2

```
=== STATYSTYKA BUFORA ===
Lacznie wstawiono: 300
Lacznie pobrano: 300
Calkowity czas wykonania: 10154 ms
Wszystkie watki zakonczone
```

Rysunek 6: Wynik po uruchomieniu programu

1.1.c n1<n2 (poprawione iteracje)

```
Run    lab3 [:PKmon.main()]
14 sec
13 sec
[10046 ms] Producer 0 put: 99
  [RESUME] Consumer 2 wznawia prace
[10046 ms] Consumer 2 got: 99
  [RESUME] Consumer 0 wznawia prace
  [WAIT] Consumer 0 czeka (buffer pusty)
  [WAIT] Consumer 1 czeka (buffer pusty)
[10055 ms] Producer 1 put: 99
  [RESUME] Consumer 0 wznawia prace
[10055 ms] Consumer 0 got: 99
  [RESUME] Consumer 1 wznawia prace
  [WAIT] Consumer 1 czeka (buffer pusty)
  [WAIT] Consumer 2 czeka (buffer pusty)
Producer 0 zakonczyl prace
Producer 1 zakonczyl prace
  [WAIT] Consumer 0 czeka (buffer pusty)
```

Rysunek 7: Wynik po uruchomieniu programu

1.1.c n1>n2

```
lab3 [:P
[15103 ms] Producer 2 put: 99
[15103 ms] Consumer 0 got: 99
[15104 ms] Consumer 1 got: 99
Producer 2 zakonczyl prace
Consumer 1 zakonczyl prace
Consumer 0 zakonczyl prace

=== STATYSTYKA BUFORA ===
Lacznie wstawiono: 300
Lacznie pobrano: 300
Calkowity czas wykonania: 15205 ms
Wszystkie watki zakonczone
```

Rysunek 8: Wynik po uruchomieniu programu

Wywołanie - widzimy częste blokowanie producentów (producenci wytwarzają dane szybciej niż konsumenci je odbierają)

```
[14391 ms] Producer 0 put: 97
[14391 ms] Consumer 1 got: 92
  [RESUME] Producer 2 wznawia prace
  [WAIT] Producer 0 czeka (buffer pelny)
[14470 ms] Consumer 0 got: 96
[14470 ms] Producer 1 put: 93
  [RESUME] Producer 0 wznawia prace
  [WAIT] Producer 1 czeka (buffer pelny)
[14493 ms] Producer 2 put: 97
[14493 ms] Consumer 1 got: 97
  [RESUME] Producer 1 wznawia prace
  [WAIT] Producer 2 czeka (buffer pelny)
[14571 ms] Producer 0 put: 98
[14571 ms] Consumer 0 got: 93
  [RESUME] Producer 2 wznawia prace
  [WAIT] Producer 2 czeka (buffer pelny)
[14593 ms] Producer 1 put: 94
[14593 ms] Consumer 1 got: 98
  [RESUME] Producer 2 wznawia prace
  [WAIT] Producer 2 czeka (buffer pelny)
[14671 ms] Producer 0 put: 99
[14671 ms] Consumer 0 got: 94
  [RESUME] Producer 2 wznawia prace
  [WAIT] Producer 0 czeka (buffer pelny)
[14694 ms] Consumer 1 got: 97
  [RESUME] Producer 0 wznawia prace
[14694 ms] Producer 1 put: 95
  [WAIT] Producer 1 czeka (buffer pelny)
[14774 ms] Producer 2 put: 98
[14774 ms] Consumer 0 got: 99
  [RESUME] Producer 1 wznawia prace
  [WAIT] Producer 2 czeka (buffer pelny)
Producer 0 zakonczyl prace
[14794 ms] Consumer 1 got: 95
  [RESUME] Producer 2 wznawia prace
[14875 ms] Producer 1 put: 96
[14875 ms] Consumer 0 got: 98
[14895 ms] Producer 2 put: 99
[14895 ms] Consumer 1 got: 96
[14975 ms] Producer 1 put: 97
  [WAIT] Producer 1 czeka (buffer pelny)
[14976 ms] Consumer 0 got: 99
  [RESUME] Producer 1 wznawia prace
Producer 2 zakonczyl prace
[14996 ms] Consumer 1 got: 97
[15076 ms] Producer 1 put: 98
  [WAIT] Consumer 0 czeka (buffer pusty)
  [RESUME] Consumer 0 wznawia prace
[15076 ms] Consumer 0 got: 98
  [WAIT] Consumer 1 czeka (buffer pusty)
[15177 ms] Producer 1 put: 99
Consumer 0 zakonczyl prace
  [RESUME] Consumer 1 wznawia prace
[15177 ms] Consumer 1 got: 99
Producer 1 zakonczyl prace
Consumer 1 zakonczyl prace
```

1.1.c n1<n2

```
  [RESUME] Consumer 1 wznawia prace
[15101 ms] Consumer 1 got: 149
Consumer 0 zakonczyl prace
Producer 0 zakonczyl prace
Producer 1 zakonczyl prace
Consumer 1 zakonczyl prace

=== STATYSTYKA BUFORA ===
Lacznie wstawiono: 300
Lacznie pobrano: 300
Calkowity czas wykonania: 15203 ms
Wszystkie watki zakonczone
```

Rysunek 9: Wynik po uruchomieniu programu

Wywołanie - widzimy częste blokowanie konsumentów (producenci wytwarzają dane wolniej niż konsumenci je odbierają)

```
[14688 ms] Consumer 2 got: 145
 [RESUME] Consumer 0 wznawia prace
 [WAIT] Consumer 0 czeka (buffer pusty)
 [WAIT] Consumer 1 czeka (buffer pusty)
[14696 ms] Producer 1 put: 145
 [RESUME] Consumer 1 wznawia prace
[14696 ms] Consumer 1 got: 145
 [RESUME] Consumer 0 wznawia prace
 [WAIT] Consumer 0 czeka (buffer pusty)
 [WAIT] Consumer 2 czeka (buffer pusty)
[14788 ms] Producer 0 put: 146
 [RESUME] Consumer 0 wznawia prace
[14788 ms] Consumer 0 got: 146
 [RESUME] Consumer 2 wznawia prace
 [WAIT] Consumer 2 czeka (buffer pusty)
 [WAIT] Consumer 1 czeka (buffer pusty)
[14796 ms] Producer 1 put: 146
 [RESUME] Consumer 2 wznawia prace
[14796 ms] Consumer 2 got: 146
 [RESUME] Consumer 1 wznawia prace
 [WAIT] Consumer 1 czeka (buffer pusty)
[14889 ms] Producer 0 put: 147
 [RESUME] Consumer 1 wznawia prace
[14889 ms] Consumer 1 got: 147
 [WAIT] Consumer 0 czeka (buffer pusty)
 [WAIT] Consumer 2 czeka (buffer pusty)
[14898 ms] Producer 1 put: 147
 [RESUME] Consumer 0 wznawia prace
[14898 ms] Consumer 0 got: 147
 [RESUME] Consumer 2 wznawia prace
 [WAIT] Consumer 2 czeka (buffer pusty)
 [WAIT] Consumer 1 czeka (buffer pusty)
[14989 ms] Producer 0 put: 148
 [RESUME] Consumer 2 wznawia prace
[14989 ms] Consumer 2 got: 148
 [RESUME] Consumer 1 wznawia prace
 [WAIT] Consumer 1 czeka (buffer pusty)
[14999 ms] Producer 1 put: 148
 [WAIT] Consumer 0 czeka (buffer pusty)
 [RESUME] Consumer 1 wznawia prace
[14999 ms] Consumer 1 got: 148
 [RESUME] Consumer 0 wznawia prace
 [WAIT] Consumer 0 czeka (buffer pusty)
[15090 ms] Producer 0 put: 149
Consumer 2 zakonczyl prace
 [RESUME] Consumer 0 wznawia prace
[15090 ms] Consumer 0 got: 149
 [WAIT] Consumer 1 czeka (buffer pusty)
[15101 ms] Producer 1 put: 149
 [RESUME] Consumer 1 wznawia prace
[15101 ms] Consumer 1 got: 149
Consumer 0 zakonczyl prace
Producer 0 zakonczyl prace
Producer 1 zakonczyl prace
Consumer 1 zakonczyl prace
```

1.2.a (n1=n2=1)

```
✓ lab3 [:PKn 792 ms
Konsument 0 pobral: 88
Konsument 0 pobral: 89
Konsument 0 pobral: 90
Konsument 0 pobral: 91
Konsument 0 pobral: 92
Konsument 0 pobral: 93
Konsument 0 pobral: 94
Konsument 0 pobral: 95
Konsument 0 pobral: 96
Konsument 0 pobral: 97
Konsument 0 pobral: 98
Konsument 0 pobral: 99

BUILD SUCCESSFUL in 649ms
2 actionable tasks: 2 executed
```

Rysunek 10: Wynik po uruchomieniu programu

1.2.b (n1=3, n2=2, iter1=100, iter2=150)

```
✓ lab3 [:PKn 695 ms
Konsument 0 pobral: 94
Konsument 0 pobral: 95
Konsument 0 pobral: 96
Konsument 0 pobral: 97
Konsument 0 pobral: 98
Konsument 0 pobral: 99

BUILD SUCCESSFUL in 591ms
2 actionable tasks: 2 executed
```

Rysunek 11: Wynik po uruchomieniu programu

2

```
Run    lab3 [:PKmon.main()]
✓ lab3 [:l 9 sec, 45 ms
[ETAP 4] Koniec
[KONSUMENT] ? slot 49 = 1549
[KONSUMENT] Koniec

=== WYNIKI ===
Czas wykonania: 7874 ms
Przepustowosc: 6,35 elem/s

BUILD SUCCESSFUL in 8s
2 actionable tasks: 2 executed
```

Rysunek 12: Wynik po uruchomieniu programu

5. Wnioski

- Prędkość obróbki w potokowym przetwarzaniu z użyciem bufora zależy od: najwolniejszego etapu (cały potok nie może być szybszy niż najwolniejsze ogniwo), od rozmiaru bufora - większy bufor pozwala na mniejsze blokowanie przy nierównomiernych czasach, liczby etapów - im więcej etapów tym program działa dłużej, czasu przetwarzania etapów oraz od kosztu synchronizacji (np. czas spędzony na wait() i notify()).
- Zarówno mechanizm monitorów (synchronized, wait(), notify()), jak i semafory są efektywnymi narzędziami do rozwiązania problemu "producenta-konsumenta". Przy dużej liczbie wątków wait() i notify() mają gorszą skalowalność związaną z budzeniem wszystkich wątków, zaś semafory dają nam w tym przypadku lepsza wydajność.
- Obserwacje z wprowadzenia Thread.sleep():
  - n1>n2: Producenci częściej się blokują (bufor pełny)
  - n1<n2: Konsumenci częściej czekają (bufor pusty)
  - n1=n2: Najbardziej zrównoważone obciążenie
- Przetwarzanie potokowe umożliwia nam równoległe przetwarzanie różnych danych w różnych etapach, dzięki czemu zwiększa przepustowość systemu. Głównym wyzwaniem jest z kolei zrównoważenie potoku - jeśli jeden etap jest wyraźnie wolniejszy od pozostałych, staje się tzw. bottleneckiem limitującym wydajnośc całego systemu. Zasadniczo każde podejście jest dobre, lecz najlepsze pod względem złożoności czasowej (choć według mnie niekoniecznie pod względem łatwości zrozumienia) będzie podejście potokowe.

6. Bibliografia

https://pl.wikipedia.org/wiki/Przetwarzanie_potokow